WHERE Subquery

Used for complex filtering logic and

makes query more flexible and dynamic.

COMPARISON OPERATORS

Used to filter data by comparing two values

Comparison Operators

=		Equal	WHERE Sales = (SELECT AVG(Sales) FROM ORDERS)
!=	<>	Not Equal	WHERE Sales != (SELECT AVG(Sales) FROM ORDERS)
>		Greater than	WHERE Sales > (SELECT AVG(Sales) FROM ORDERS)
<		Less than	WHERE Sales < (SELECT AVG(Sales) FROM ORDERS)
>=		Greater than or equal to	WHERE Sales >= (SELECT AVG(Sales) FROM ORDERS)
<=		Less than or equal to	WHERE Sales <= (SELECT AVG(Sales) FROM ORDERS)

Subquery in WHERE Clause
Comparison Operators

Main Query

```
SELECT column1, column2,...
FROM   table1
WHERE  column = ( SELECT column FROM table2 WHERE condition )
```

Subquery

Subquery in WHERE Clause
Comparison Operators

Main Query

```
SELECT column1, column2,...
FROM   table1
WHERE  column = ( SELECT column FROM table2 WHERE condition )
```

Subquery

Rules

Only Scalar Subqueries are allowed to be used

```sql
-- Find the products that have a price higher than the average price of all products

-- Main Query
SELECT
ProductID,
Price,
(SELECT AVG(Price) FROM Sales.Products) AvgPrice
FROM Sales.Products
WHERE Price > (SELECT AVG(Price) FROM Sales.Products)
```

	ProductID	Price	AvgPrice
1	104	25	20
2	105	30	20

Products

ProductID	Product	Category	Price
101	Bottle	Accessories	10
102	Tire	Accessories	15
103	Socks	Clothing	20
104	Caps	Clothing	25
105	Gloves	Clothing	30

Original Query

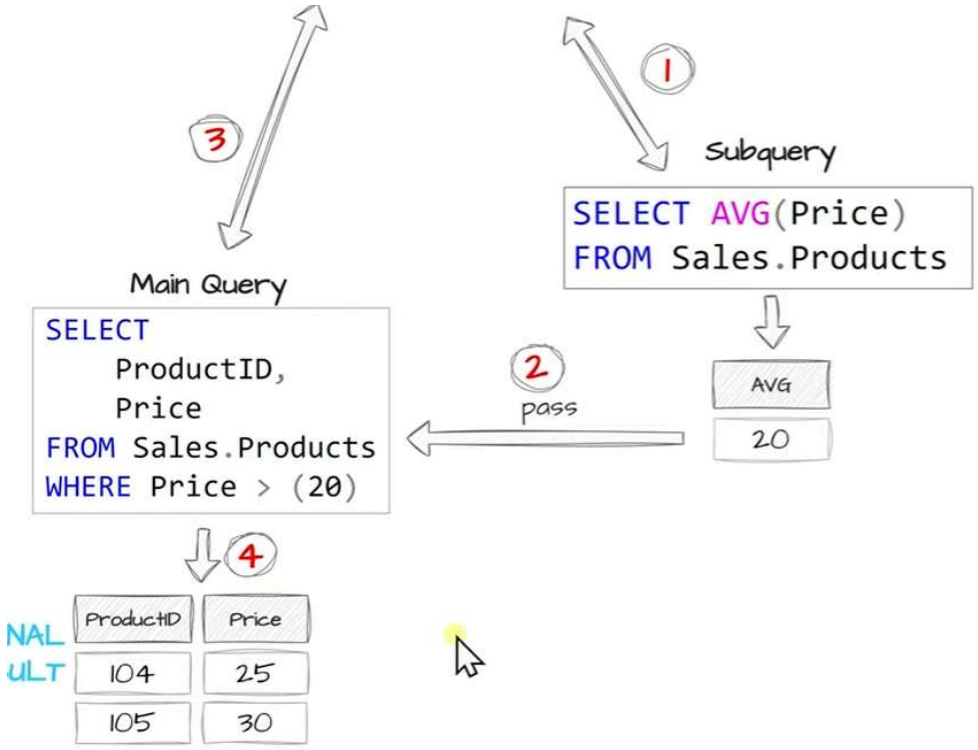
```sql
SELECT
    ProductID,
    Price
FROM Sales.Products
WHERE Price >
    (SELECT AVG(Price)
     FROM Sales.Products)
```

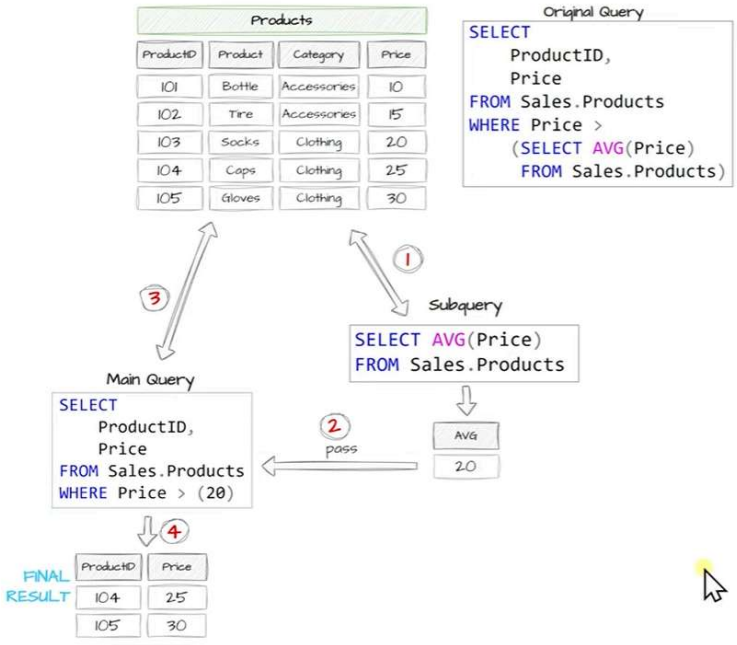
Subquery

```sql
SELECT AVG(Price)
FROM Sales.Products
```

Main Query

```sql
SELECT
```


1
Subquery
SELECT AVG(Price)
FROM Sales.Products
AVG
20
2
pass
3
Main Query
SELECT
ProductID,
Price
FROM Sales.Products
WHERE Price > (20)
4
NAL
ULT
ProductID
Price
104
25
105
30


Products
ProductID
Product
Category
Price
101
Bottle
Accessories
10
102
Tire
Accessories
15
103
Socks
Clothing
20
104
Caps
Clothing
25
105
Gloves
Clothing
30
Original Query
SELECT
ProductID,
Price
FROM Sales.Products
WHERE Price >
(SELECT AVG(Price)
FROM Sales.Products)
1
Subquery
SELECT AVG(Price)
FROM Sales.Products
AVG
20
2
pass
3
Main Query
SELECT
ProductID,
Price
FROM Sales.Products
WHERE Price > (20)
4
FINAL
RESULT
ProductID
Price
104
25
105
30

FILTERTING (SINGLE VALUE)

```sql
SELECT
*
FROM Sales.Orders
WHERE CustomerID = 1
```

FILTERTING (LIST OF VALUES)

```sql
SELECT
*
FROM Sales.Orders
WHERE CustomerID IN (1,2,3,4)
```

IN OPERATOR

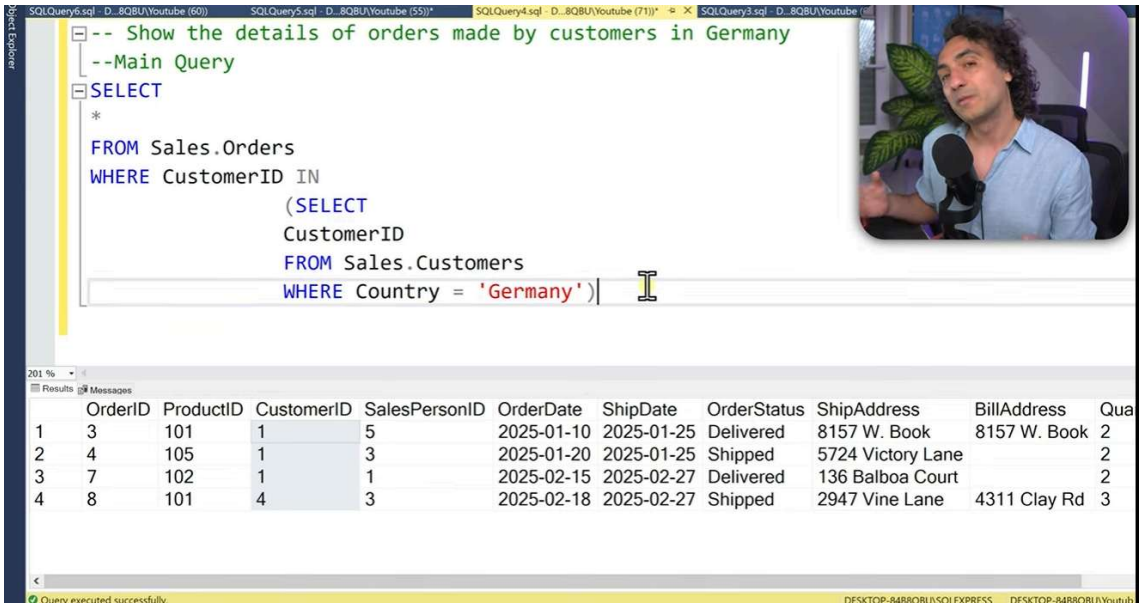
Checks whether a value matches any value from a list.

Subquery in
WHERE Clause
In Operator

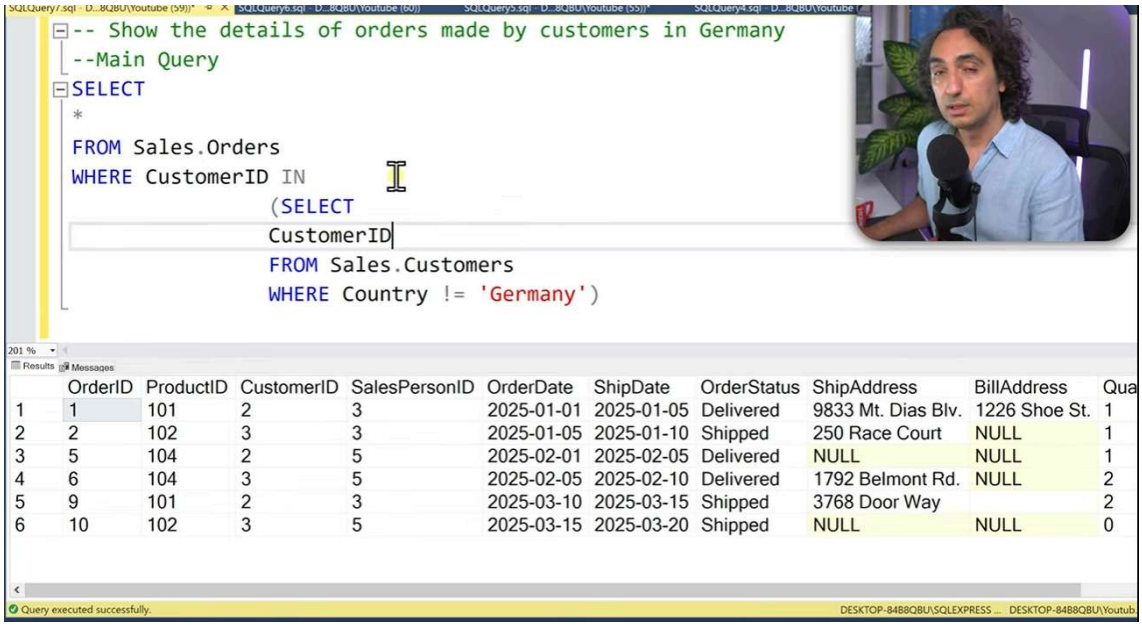
Main Query

```sql
SELECT column1, column2,...
FROM   table1
WHERE  column IN ( SELECT column FROM table2 WHERE condition )
```

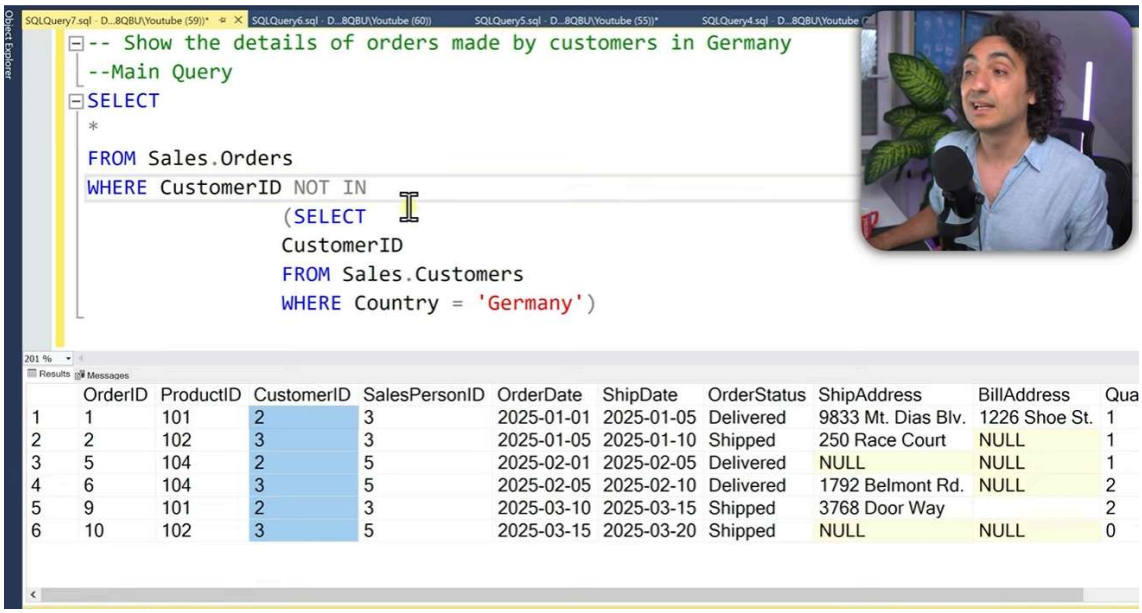
Subquery



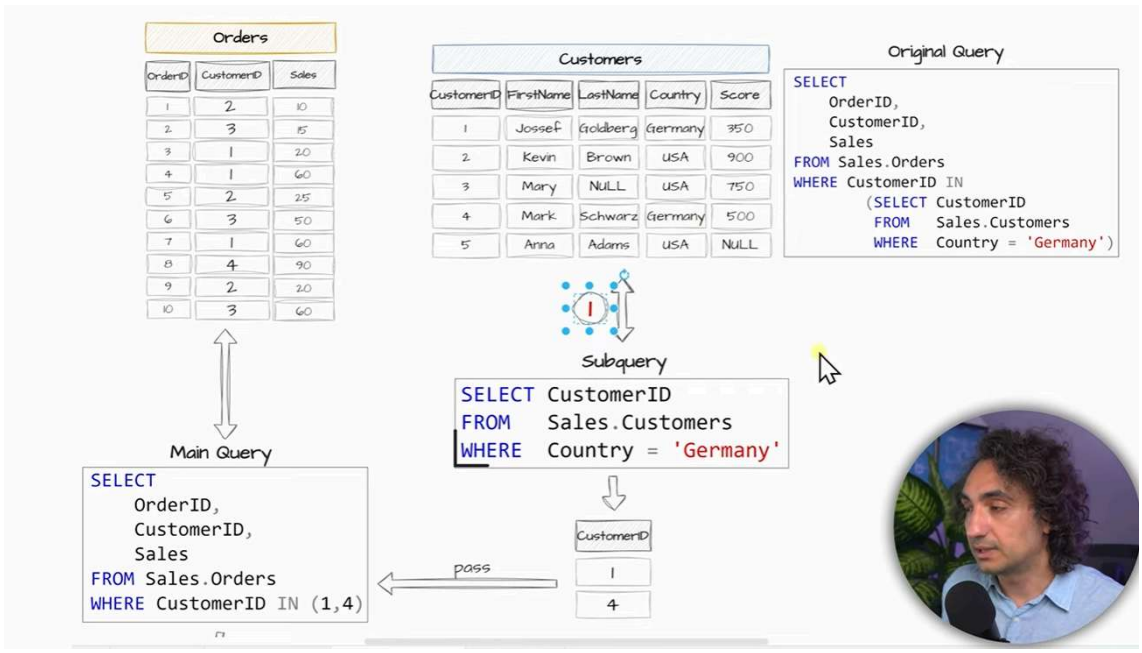
	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Qua
1	3	101	1	5	2025-01-10	2025-01-25	Delivered	8157 W. Book	8157 W. Book	2
2	4	105	1	3	2025-01-20	2025-01-25	Shipped	5724 Victory Lane		2
3	7	102	1	1	2025-02-15	2025-02-27	Delivered	136 Balboa Court		2
4	8	101	4	3	2025-02-18	2025-02-27	Shipped	2947 Vine Lane	4311 Clay Rd	3

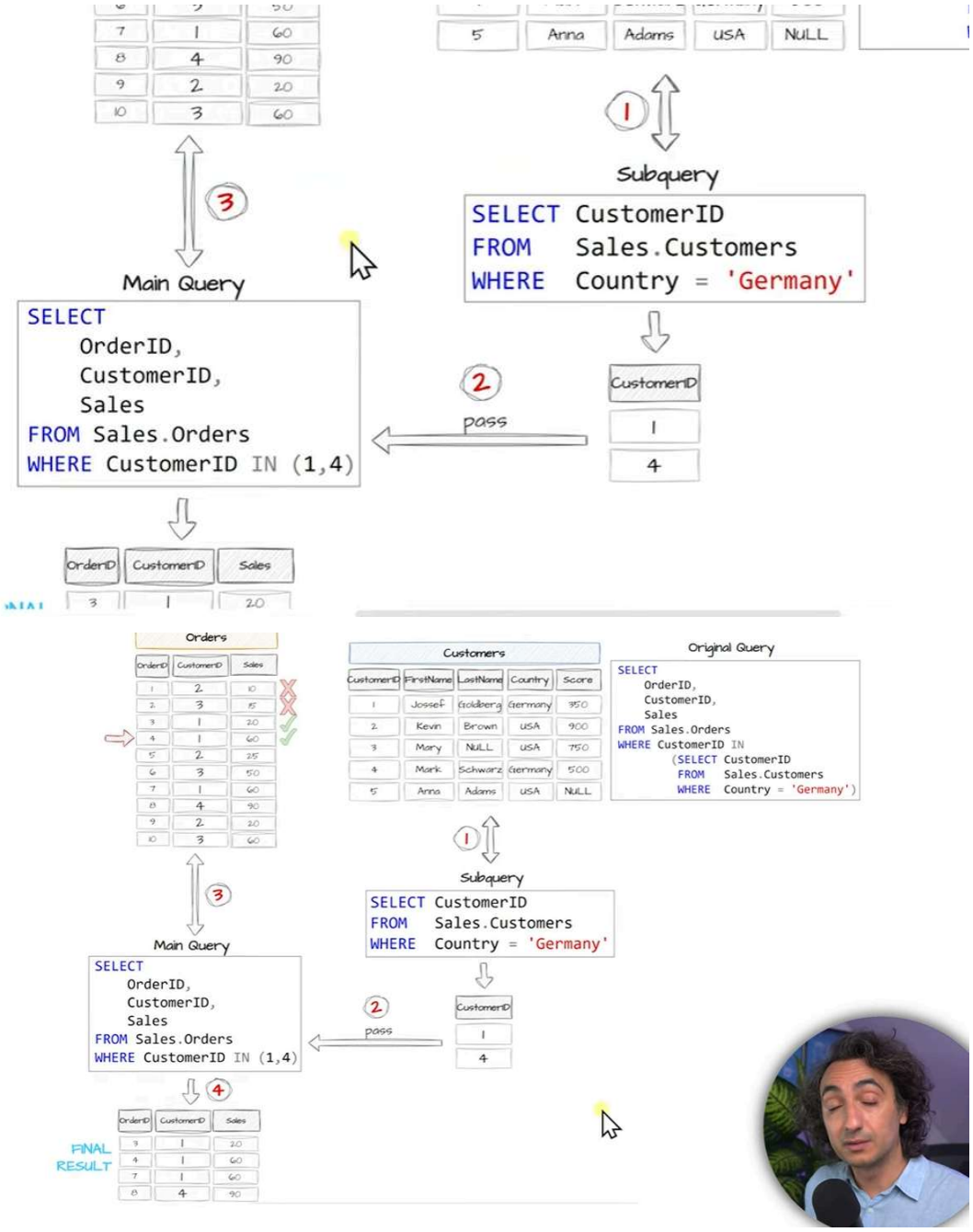


	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Qua
1	1	101	2	3	2025-01-01	2025-01-05	Delivered	9833 Mt. Dias Blv.	1226 Shoe St.	1
2	2	102	3	3	2025-01-05	2025-01-10	Shipped	250 Race Court	NULL	1
3	5	104	2	5	2025-02-01	2025-02-05	Delivered	NULL	NULL	1
4	6	104	3	5	2025-02-05	2025-02-10	Delivered	1792 Belmont Rd.	NULL	2
5	9	101	2	3	2025-03-10	2025-03-15	Shipped	3768 Door Way		2
6	10	102	3	5	2025-03-15	2025-03-20	Shipped	NULL	NULL	0



	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Qua
1	1	101	2	3	2025-01-01	2025-01-05	Delivered	9833 Mt. Dias Blv.	1226 Shoe St.	1
2	2	102	3	3	2025-01-05	2025-01-10	Shipped	250 Race Court	NULL	1
3	5	104	2	5	2025-02-01	2025-02-05	Delivered	NULL	NULL	1
4	6	104	3	5	2025-02-05	2025-02-10	Delivered	1792 Belmont Rd.	NULL	2
5	9	101	2	3	2025-03-10	2025-03-15	Shipped	3768 Door Way		2
6	10	102	3	5	2025-03-15	2025-03-20	Shipped	NULL	NULL	0




7 1 60
8 4 90
9 2 20
10 3 60
5 Anna Adams USA NULL
1
Subquery
SELECT CustomerID
FROM Sales.Customers
WHERE Country = 'Germany'
3
Main Query
SELECT
OrderID,
CustomerID,
Sales
FROM Sales.Orders
WHERE CustomerID IN (1,4)
2
pass
CustomerID
1
4
OrderID CustomerID Sales
Orders
Customers
CustomerID FirstName LastName Country Score
1 Jossef Goldberg Germany 350
2 Kevin Brown USA 900
3 Mary NULL USA 750
4 Mark Schwarz Germany 500
5 Anna Adams USA NULL
Original Query
SELECT
OrderID,
CustomerID,
Sales
FROM Sales.Orders
WHERE CustomerID IN
(SELECT CustomerID
FROM Sales.Customers
WHERE Country = 'Germany')
FINAL RESULT

Subquery

ANY | ALL

CASE WHEN Sales > 5

ANY OPERATOR

Checks if a value matches ANY value within a list.

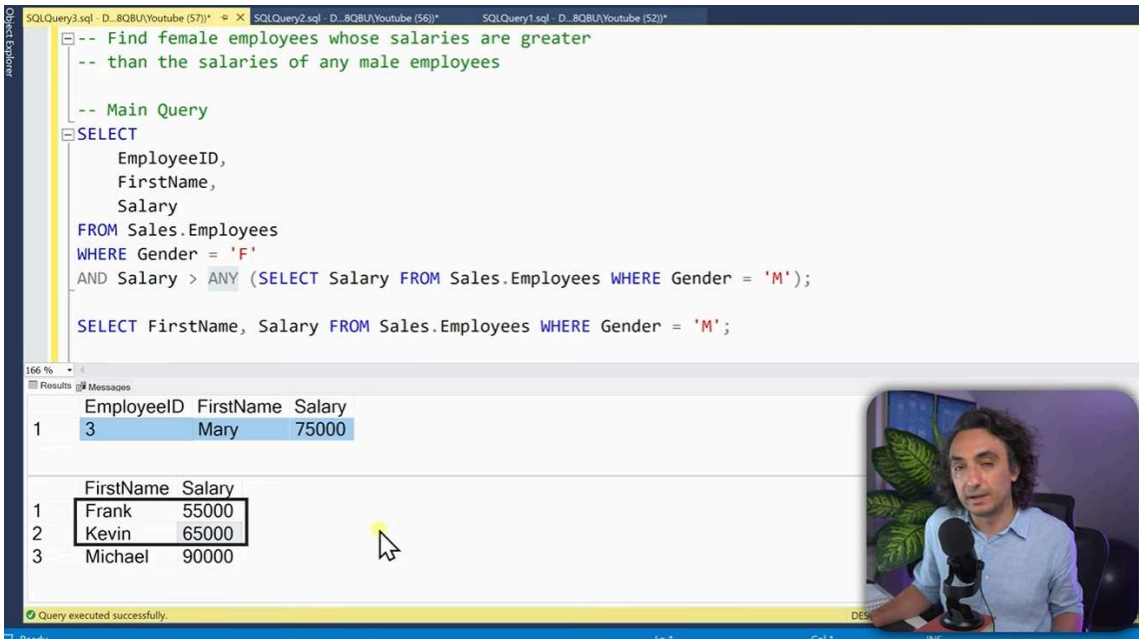
Used to check if a value is true for AT LEAST one of the values in a list.

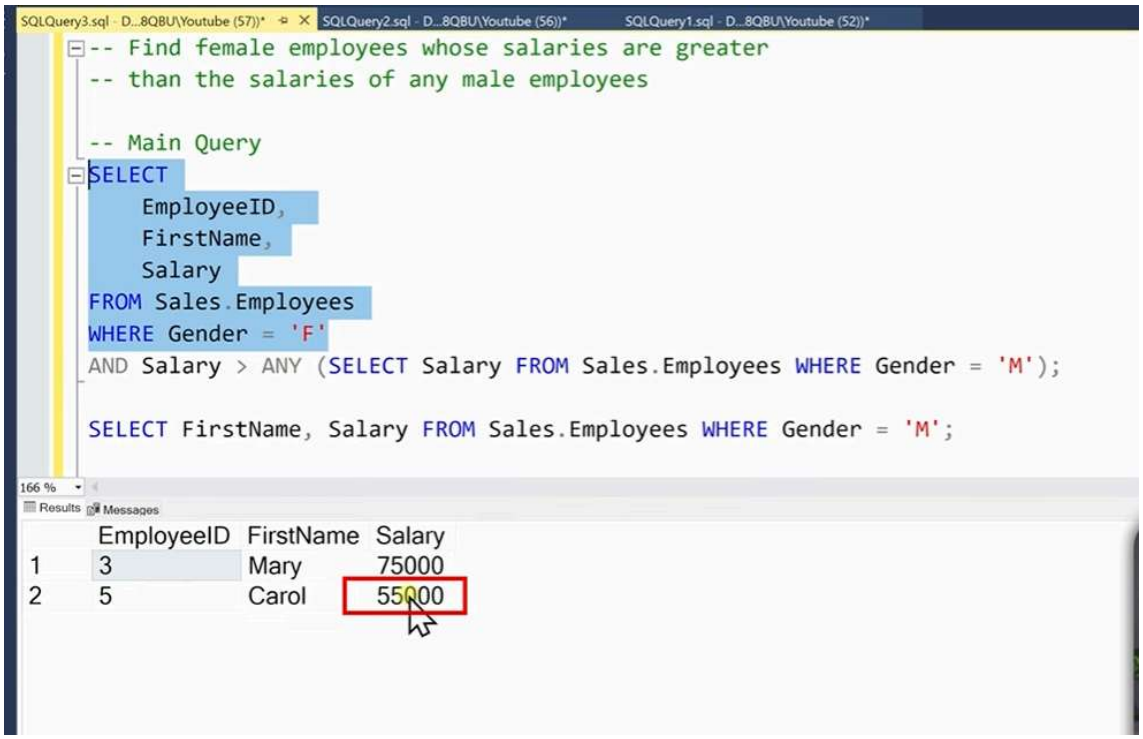
Subquery in
WHERE Clause
ALL Operator

Main Query

```
SELECT column1, column2,...
FROM   table1
WHERE  column < ALL( SELECT column FROM table1 WHERE condition )
```

Subquery


SQLQuery3.sql - D...8QBU\Youtube (57))*
SQLQuery2.sql - D...8QBU\Youtube (56))*
SQLQuery1.sql - D...8QBU\Youtube (52))*
-- Find female employees whose salaries are greater
-- than the salaries of any male employees
-- Main Query
SELECT
EmployeeID,
FirstName,
Salary
FROM Sales.Employees
WHERE Gender = 'F'
AND Salary > ANY (SELECT Salary FROM Sales.Employees WHERE Gender = 'M');
SELECT FirstName, Salary FROM Sales.Employees WHERE Gender = 'M';
166 %
Results
Messages
EmployeeID FirstName Salary
1 3 Mary 75000
FirstName Salary
1 Frank 55000
2 Kevin 65000
3 Michael 90000
Query executed successfully.

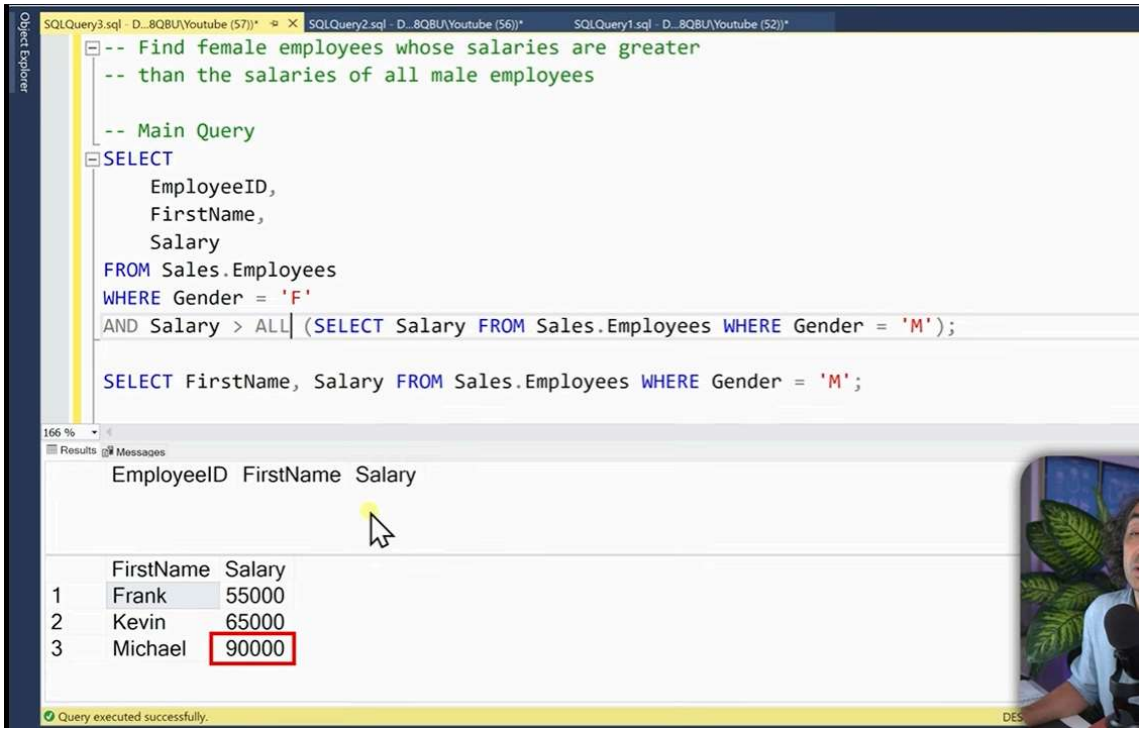

SQLQuery3.sql - D...8QBU\Youtube (57))*
SQLQuery2.sql - D...8QBU\Youtube (56))*
SQLQuery1.sql - D...8QBU\Youtube (52))*
-- Find female employees whose salaries are greater
-- than the salaries of any male employees
-- Main Query
SELECT
EmployeeID,
FirstName,
Salary
FROM Sales.Employees
WHERE Gender = 'F'
AND Salary > ANY (SELECT Salary FROM Sales.Employees WHERE Gender = 'M');
SELECT FirstName, Salary FROM Sales.Employees WHERE Gender = 'M';
166 %
Results
Messages
EmployeeID FirstName Salary
1 3 Mary 75000
2 5 Carol 55000

ALL OPERATOR

Checks if a value matches ALL values within a list.

SQL TASK

Find female employees whose salaries are greater than the salaries of all male employees

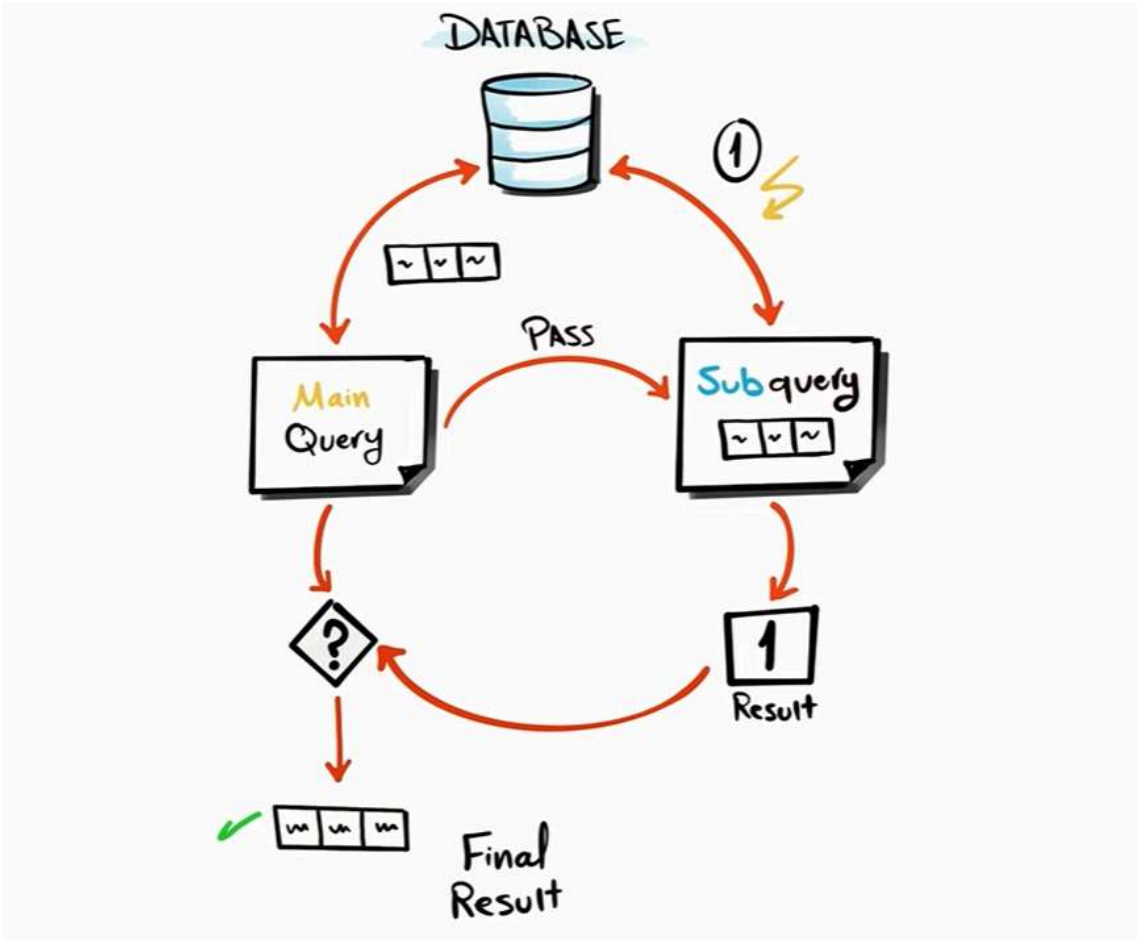


NON-CORRELATED SUBQUERY

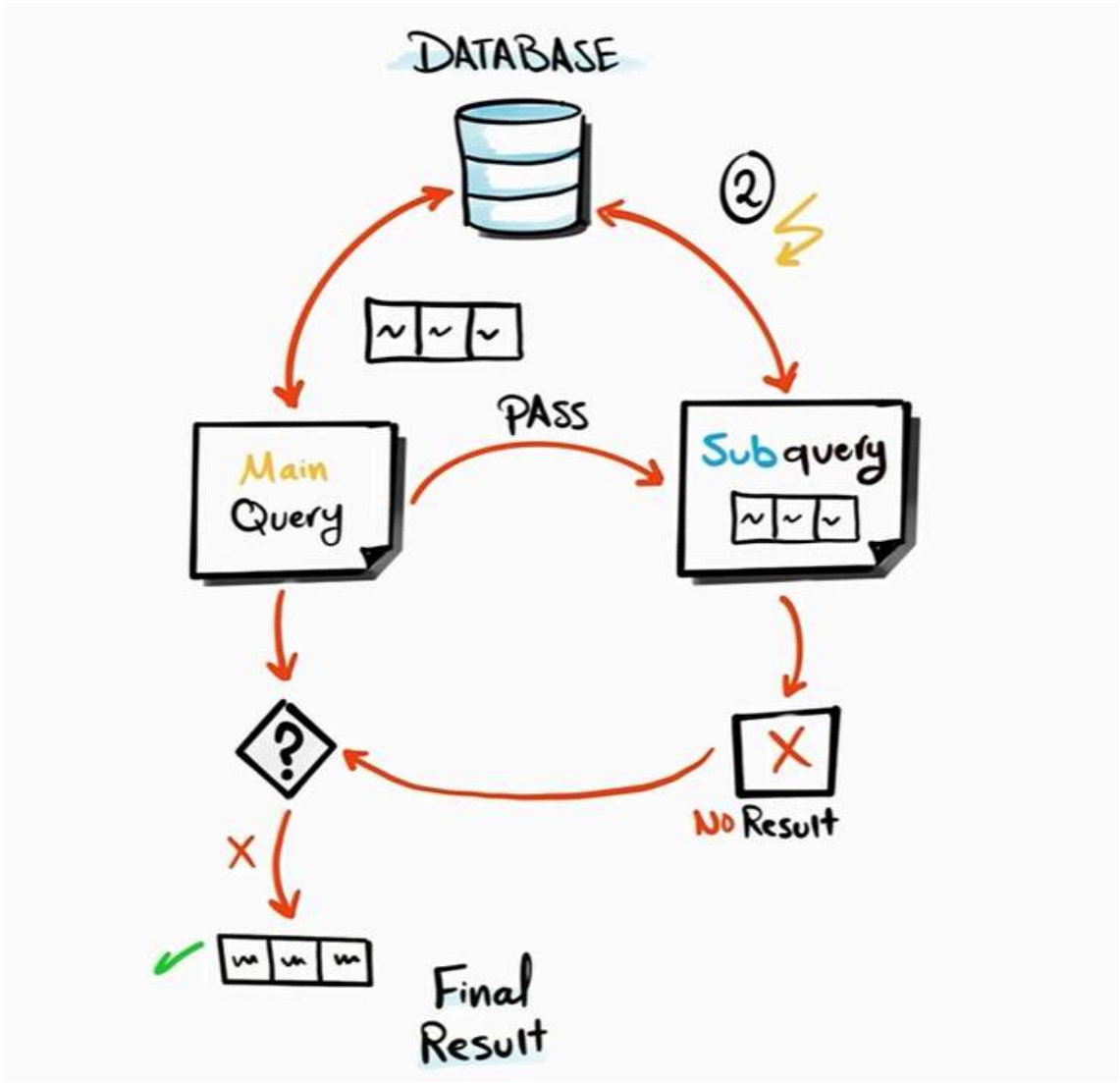
A Subquery that can run independtly from the Main Query

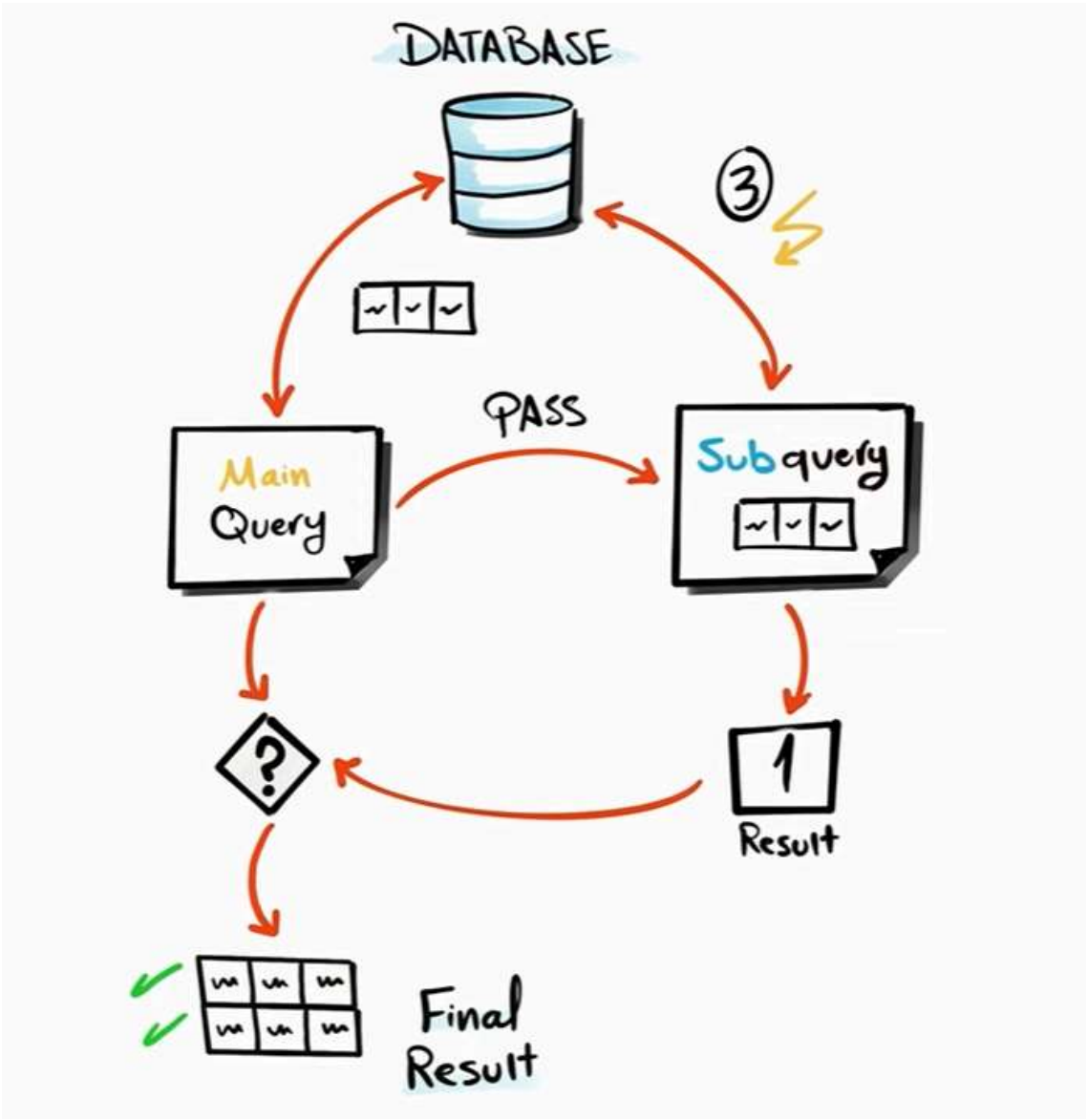
CORRELATED SUBQUERY

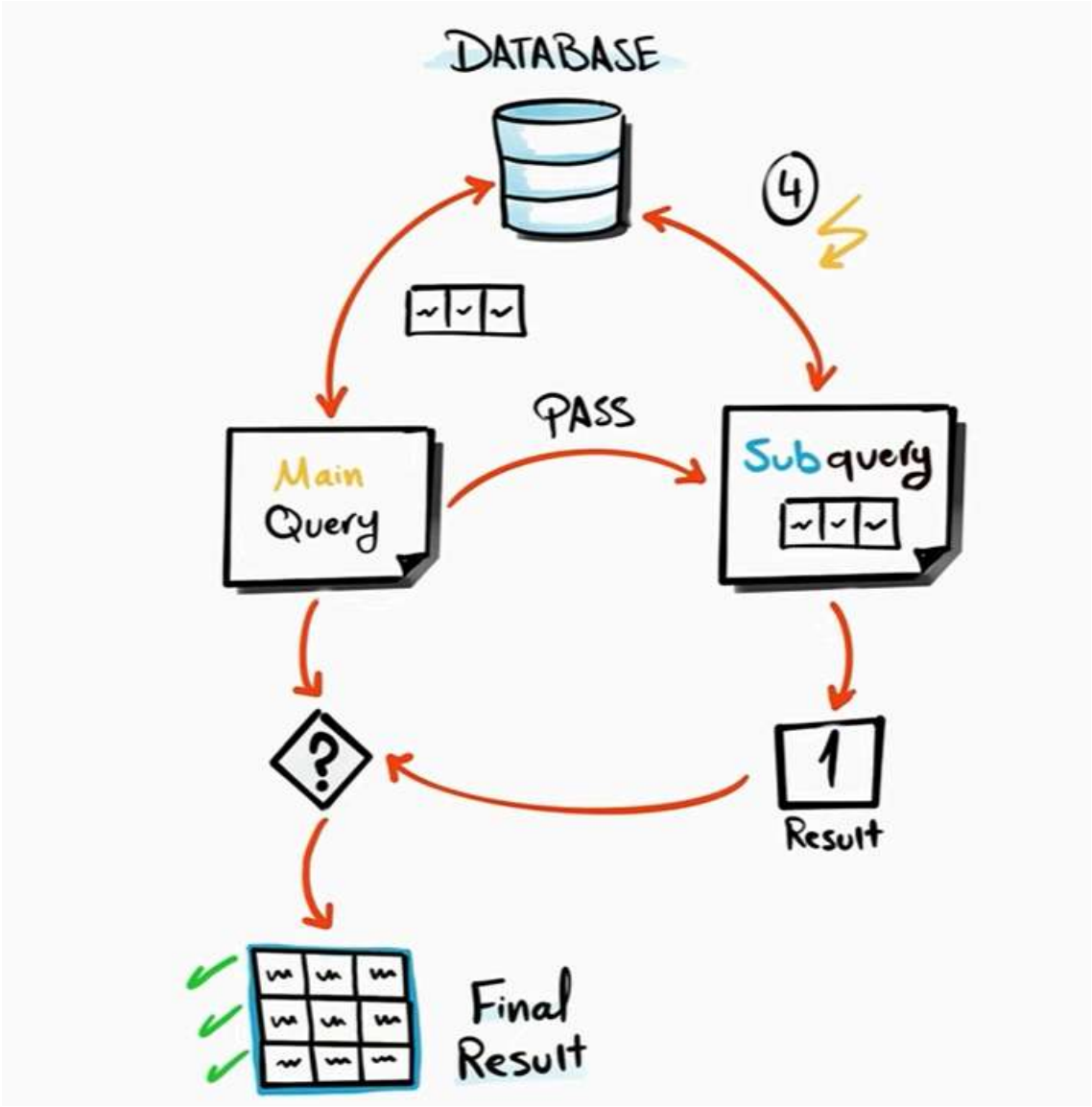
A Subquery that relays on values from the Main Query



This process works row by row.

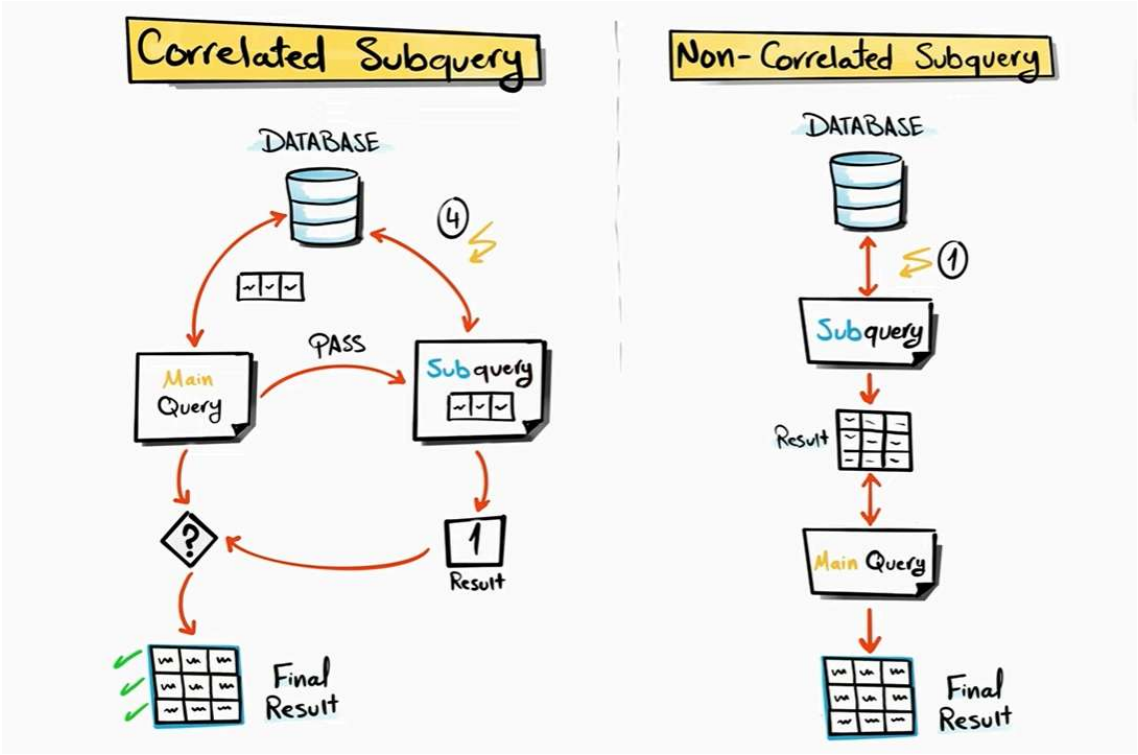

DATABASE
2
PASS
Main Query
Subquery
?
X
No Result
X
Final Result


DATABASE
3
PASS
Main
Query
Subquery
?
1
Result
Final
Result



Corrorelated subquery Depending on main query. Subquery executed for each row that's we gona get from main query so as an example we have four rows in main query and subquery executed four times for each row.

Iteration of each row happened that gona be retrieved from the main query for further expandation.



NO iterations in non - correlated subquery.
Query executed only onces and not depend on the main query.

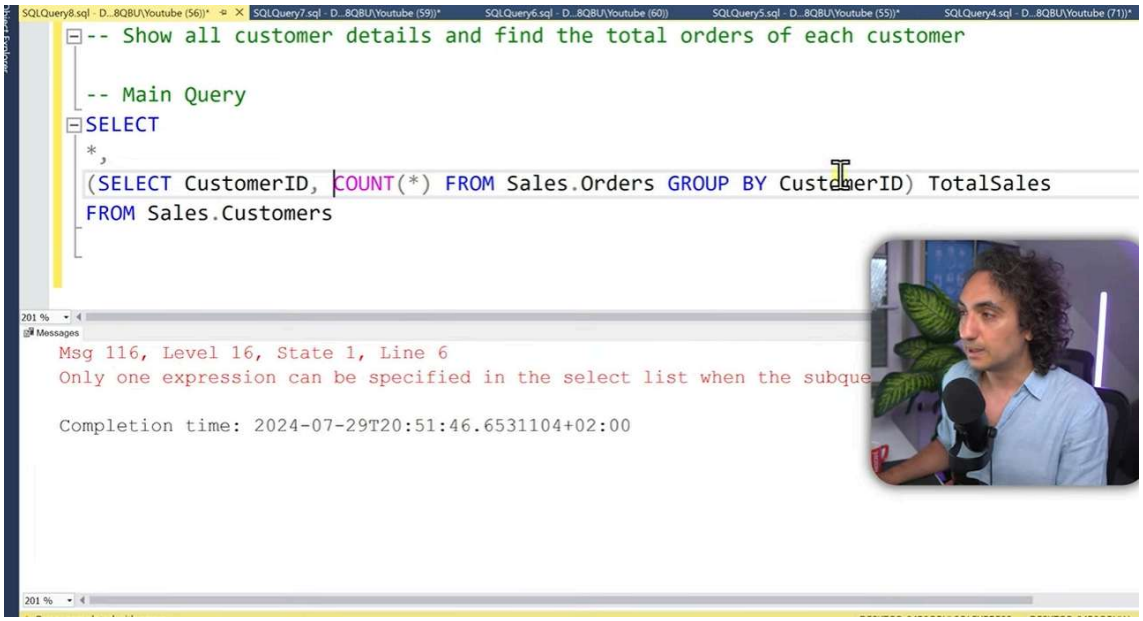
```
-- Show all customer details and find the total orders of each customer

-- Main Query
SELECT
*,
(SELECT COUNT(*) FROM Sales.Orders) TotalSales
FROM Sales.Customers
```

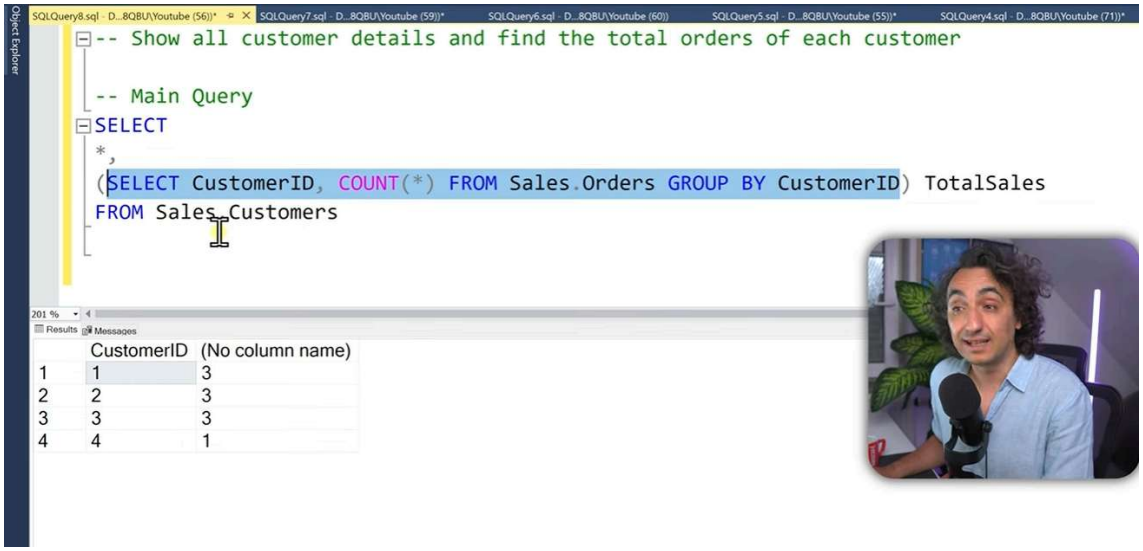
	CustomerID	FirstName	LastName	Country	Score	TotalSales
1	1	Jossef	Goldberg	Germany	350	10
2	2	Kevin	Brown	USA	900	10
3	3	Mary	NULL	USA	750	10
4	4	Mark	Schwarz	Germany	500	10
5	5	Anna	Adams	USA	NULL	10

Query executed successfully.

Each customer will have different total order

We can not used group by subquery in select clause because it will return multiple records and we can used only scalar subquery in a select clause.





We can used now correlated subquery
Current query is totally non- correlated query.
Now we told query execute for specific customers not for the table.

```sql
-- Show all customer details and find the total orders of each customer

-- Main Query
SELECT
*,
(SELECT COUNT(*) FROM Sales.Orders o WHERE o.CustomerID = c.CustomerID) TotalSales
FROM Sales.Customers c
```

	CustomerID	FirstName	LastName	Country	Score	TotalSales
1	1	Jossef	Goldberg	Germany	350	3
2	2	Kevin	Brown	USA	900	3
3	3	Mary	NULL	USA	750	3
4	4	Mark	Schwarz	Germany	500	1
5	5	Anna	Adams	USA	NULL	0

For each row subquery will be executed

```sql
-- Show all customer details and find the total orders of each customer

-- Main Query
SELECT
*,
(SELECT COUNT(*) FROM Sales.Orders o WHERE o.CustomerID = 1) TotalSales
FROM Sales.Customers c
```

	(No column name)
1	3

	Non-Correlated Subquery	Correlated Subquery
Definition	Subquery is independent of the main query	Subquery is dependent of the main query
Execution	Executed once and its result is used by the main query Can be executed on its Own	Executed for each row processed by the main query Can't be executed on its Own.
Easy to use	Easier to read	Harder to read and more complex
Performance	Executed only once leads to better Performance	Executed multiple times leads to bad Performance
Usage	Static Comparisons, Filtering with Constants	Row-by-Row Comparisons, Dynamic Filtering

EXISTS

Check if a subquery returns any rows

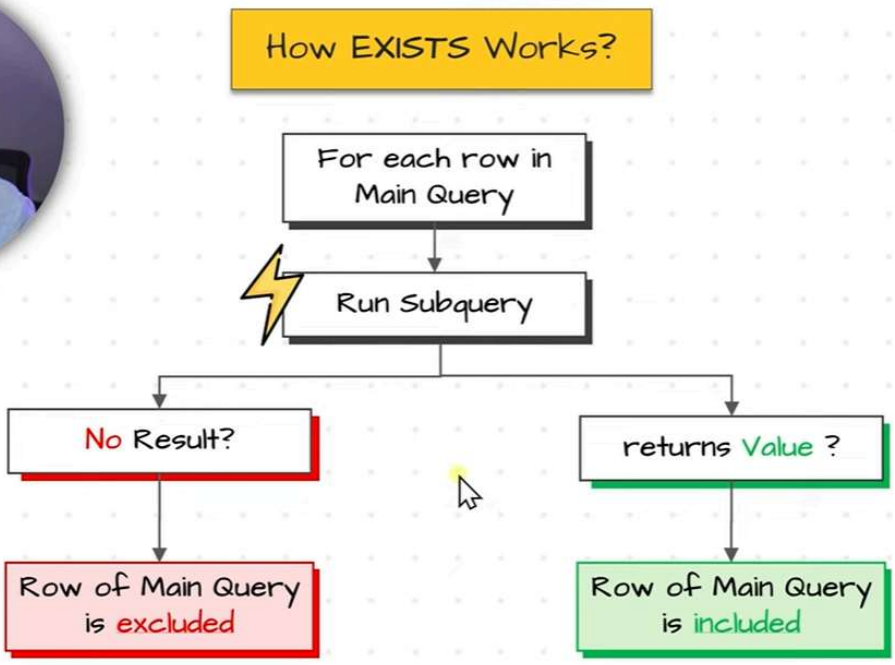
Correlated Subquery in WHERE Clause EXISTS Operator

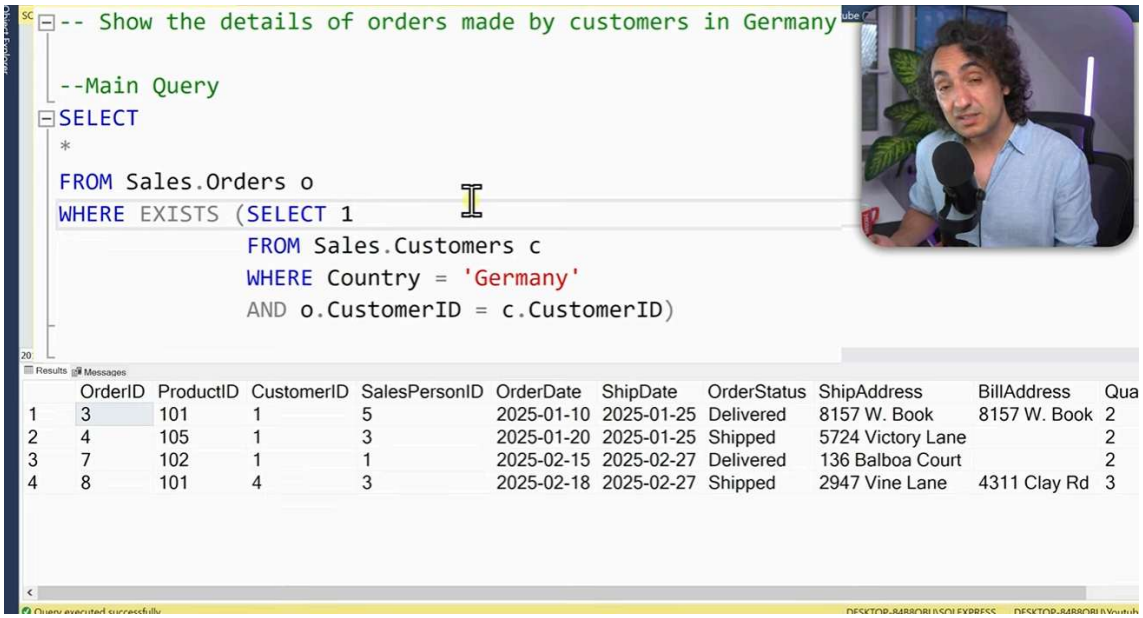
Main Query

```
SELECT column1, column2,...
FROM   Table2
WHERE  EXISTS (  SELECT 1
                 FROM Table1
                 WHERE Table1.ID = Table2.ID  )
```

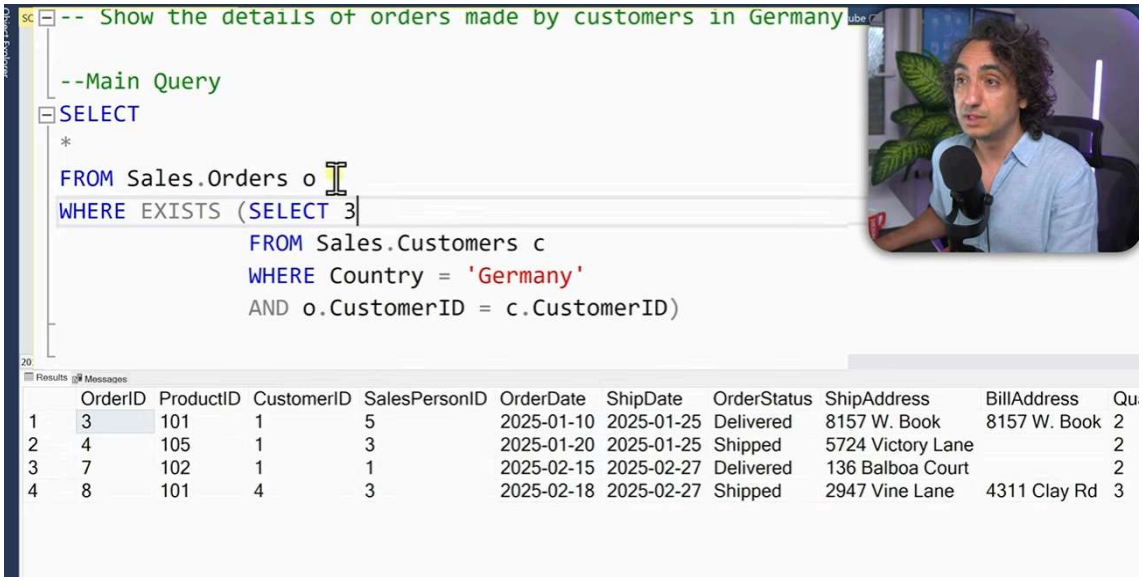
Subquery

How EXISTS Works?

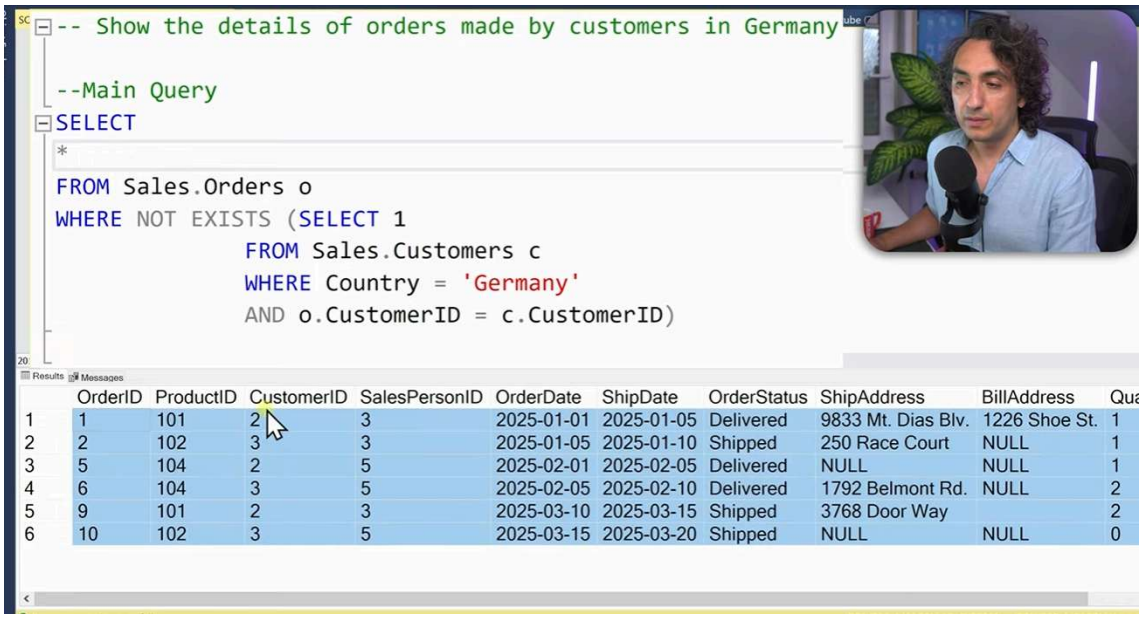




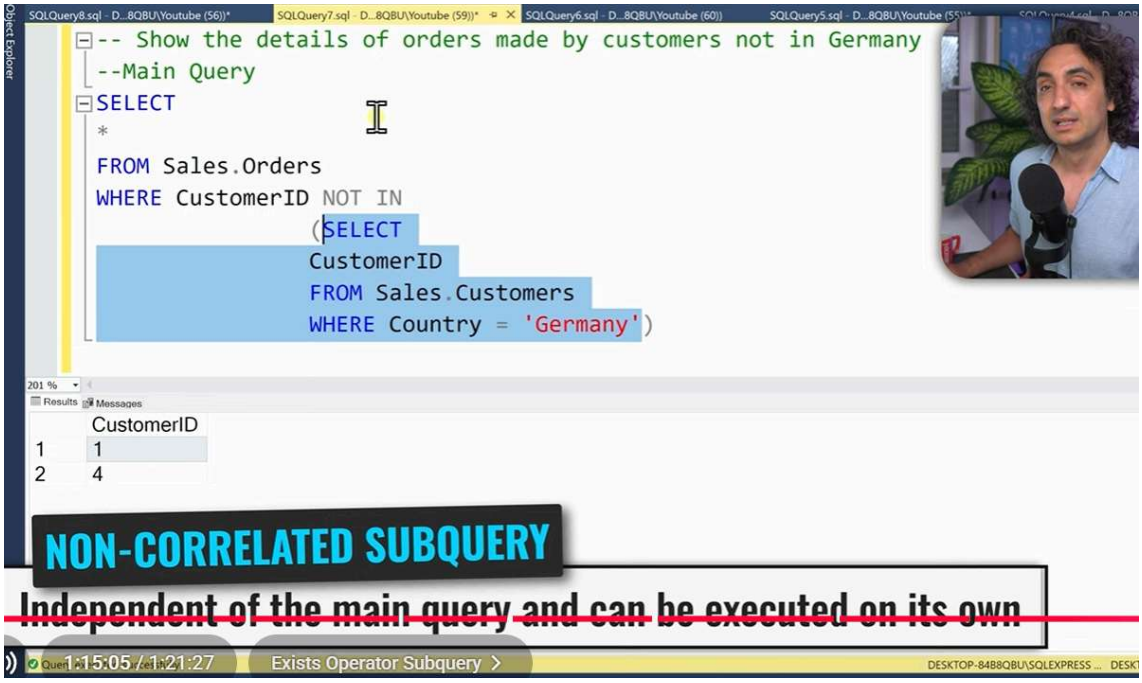
	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Qua
1	3	101	1	5	2025-01-10	2025-01-25	Delivered	8157 W. Book	8157 W. Book	2
2	4	105	1	3	2025-01-20	2025-01-25	Shipped	5724 Victory Lane		2
3	7	102	1	1	2025-02-15	2025-02-27	Delivered	136 Balboa Court		2
4	8	101	4	3	2025-02-18	2025-02-27	Shipped	2947 Vine Lane	4311 Clay Rd	3



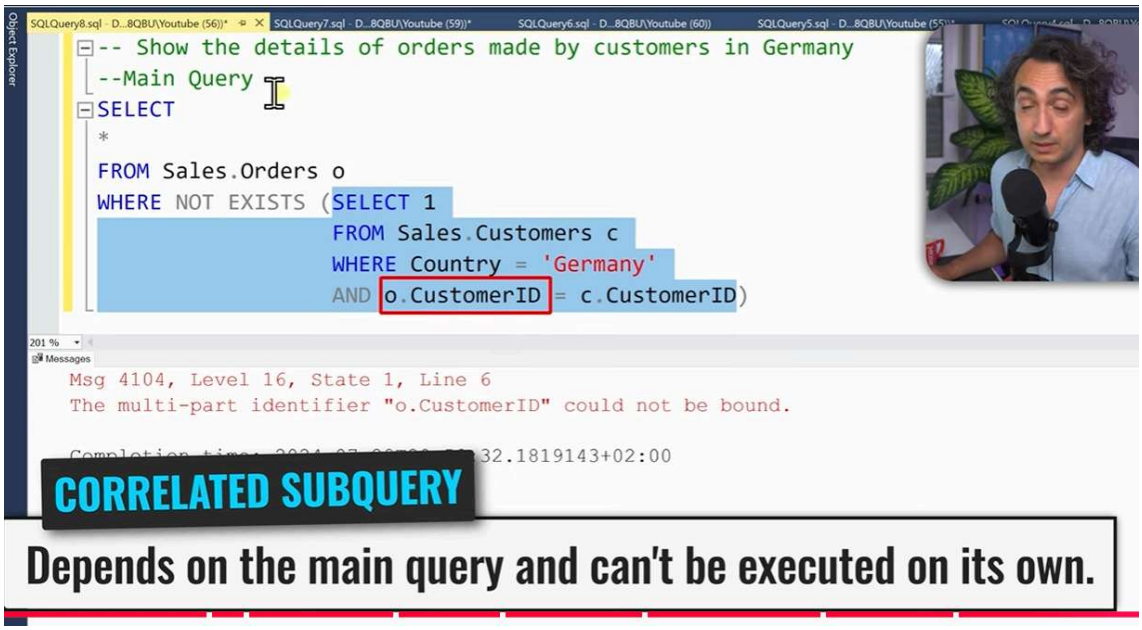
	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Qu
1	3	101	1	5	2025-01-10	2025-01-25	Delivered	8157 W. Book	8157 W. Book	2
2	4	105	1	3	2025-01-20	2025-01-25	Shipped	5724 Victory Lane		2
3	7	102	1	1	2025-02-15	2025-02-27	Delivered	136 Balboa Court		2
4	8	101	4	3	2025-02-18	2025-02-27	Shipped	2947 Vine Lane	4311 Clay Rd	3

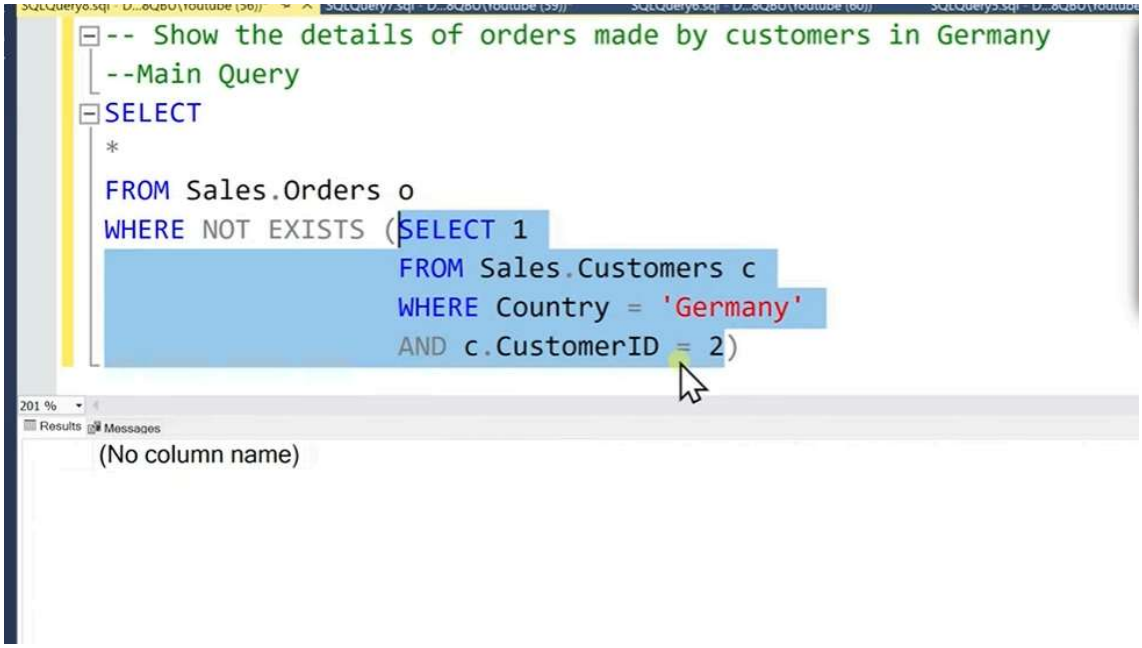


	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Qua
1	1	101	2	3	2025-01-01	2025-01-05	Delivered	9833 Mt. Dias Blv.	1226 Shoe St.	1
2	2	102	3	3	2025-01-05	2025-01-10	Shipped	250 Race Court	NULL	1
3	5	104	2	5	2025-02-01	2025-02-05	Delivered	NULL	NULL	1
4	6	104	3	5	2025-02-05	2025-02-10	Delivered	1792 Belmont Rd.	NULL	2
5	9	101	2	3	2025-03-10	2025-03-15	Shipped	3768 Door Way		2
6	10	102	3	5	2025-03-15	2025-03-20	Shipped	NULL	NULL	0

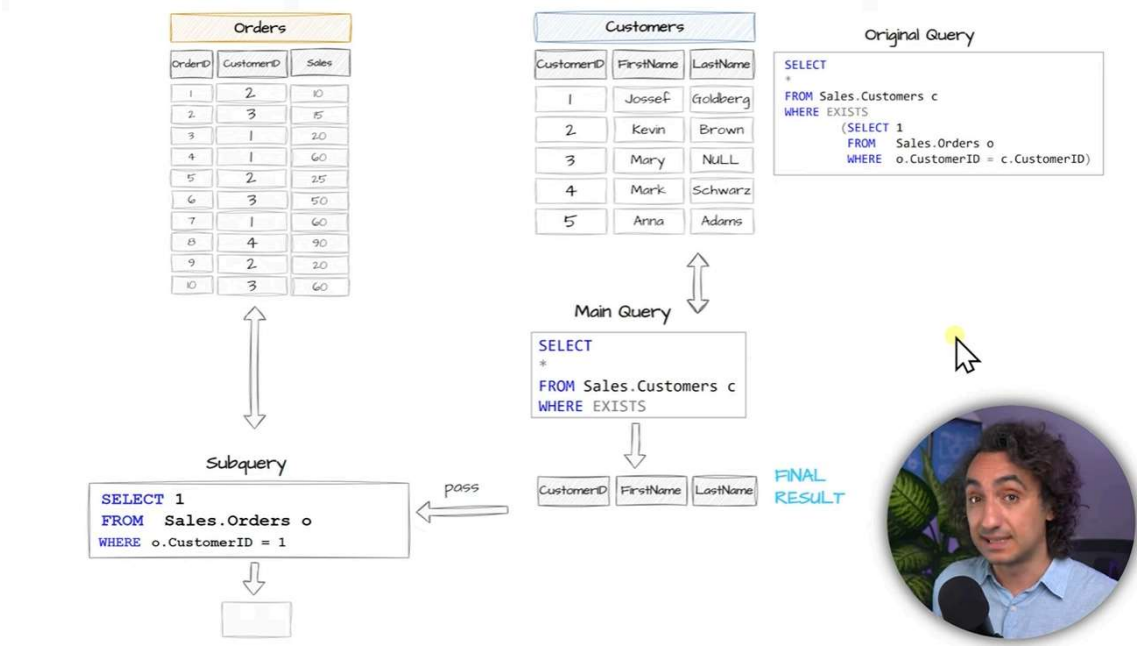


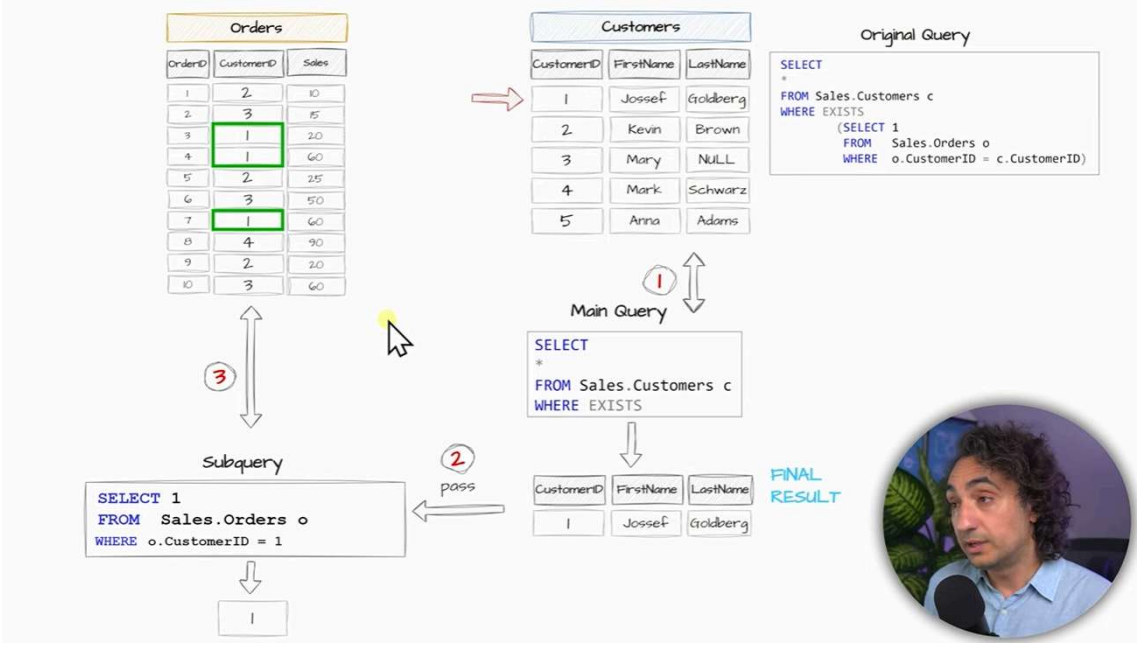
	CustomerID
1	1
2	4





Correlated query is hard to understand and we can not run independently their subquery to test result.


Orders
OrderID CustomerID Sales
1 2 10
2 3 15
3 1 20
4 1 60
5 2 25
6 3 50
7 1 60
8 4 90
9 2 20
10 3 60
Customers
CustomerID FirstName LastName
1 Jossef Goldberg
2 Kevin Brown
3 Mary NULL
4 Mark Schwarz
5 Anna Adams
Original Query
SELECT
*
FROM Sales.Customers c
WHERE EXISTS
(SELECT 1
FROM Sales.Orders o
WHERE o.CustomerID = c.CustomerID)
Main Query
SELECT
*
FROM Sales.Customers c
WHERE EXISTS
Subquery
SELECT 1
FROM Sales.Orders o
WHERE o.CustomerID = 1
pass
CustomerID FirstName LastName
FINAL RESULT


Orders
OrderID CustomerID Sales
1 2 10
2 3 15
3 1 20
4 1 60
5 2 25
6 3 50
7 1 60
8 4 90
9 2 20
10 3 60
Customers
CustomerID FirstName LastName
1 Jossef Goldberg
2 Kevin Brown
3 Mary NULL
4 Mark Schwarz
5 Anna Adams
Original Query
SELECT
*
FROM Sales.Customers c
WHERE EXISTS
(SELECT 1
FROM Sales.Orders o
WHERE o.CustomerID = c.CustomerID)
1
Main Query
SELECT
*
FROM Sales.Customers c
WHERE EXISTS
3
Subquery
SELECT 1
FROM Sales.Orders o
WHERE o.CustomerID = 1
2
pass
1
CustomerID FirstName LastName
1 Jossef Goldberg
FINAL RESULT

Orders

OrderID	CustomerID	Sales
1	2	10
2	3	15
3	1	20
4	1	60
5	2	25
6	3	50
7	1	60
8	4	90
9	2	20
10	3	60

Customers

CustomerID	FirstName	LastName
1	Josef	Goldberg
2	Kevin	Brown
3	Mary	NULL
4	Mark	Schwarz
5	Anna	Adams

Original Query

```
SELECT
*
FROM Sales.Customers c
WHERE EXISTS
        (SELECT 1
         FROM   Sales.Orders o
         WHERE  o.CustomerID = c.CustomerID)
```

1 Main Query

```
SELECT
*
FROM Sales.Customers c
WHERE EXISTS
```

3

Subquery

```
SELECT 1
FROM  Sales.Orders o
WHERE o.CustomerID = 5
```

2 pass

CustomerID	FirstName	LastName	FINAL RESULT
1	Josef	Goldberg	✓
2	Kevin	Brown	✓
3	Mary	NULL	✓
4	Mark	Schwarz	✓
5	Anna	Adams	✗

Orders

OrderID	CustomerID	Sales
1	2	10
2	3	15
3	1	20
4	1	60
5	2	25
6	3	50
7	1	60
8	4	90
9	2	20
10	3	60

Customers

CustomerID	FirstName	LastName
1	Josef	Goldberg
2	Kevin	Brown
3	Mary	NULL
4	Mark	Schwarz
5	Anna	Adams

Original Query

```
SELECT
*
FROM Sales.Customers c
WHERE EXISTS
        (SELECT 1
         FROM   Sales.Orders o
         WHERE  o.CustomerID = c.CustomerID)
```

1 Main Query

```
SELECT
*
FROM Sales.Customers c
WHERE EXISTS
```

3

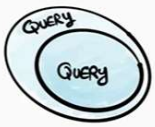
Subquery

```
SELECT 1
FROM  Sales.Orders o
WHERE o.CustomerID = 5
```

2 pass

CustomerID	FirstName	LastName	FINAL RESULT
1	Josef	Goldberg	✓
2	Kevin	Brown	✓
3	Mary	NULL	✓
4	Mark	Schwarz	✓

SUBQUERY



Query inside Another Query

Breaks Complex query into Smaller, manageable pieces.

USE CASES

- Create Temporary Result Set.
- Prepare Data Befor Joining Tables.
- Dynamic & Complex Filtering.
- Check the Existence of Rows From another Table. (EXISTS)
- Row By Row Comparison - Correlated Subquery -